

```
# Cell 1: Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

✓ Mount Google Drive and Extract Dataset

```
# Cell 2: Extract dataset
import zipfile
import os

# Path to your uploaded zip file
zip_path = '/content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/dataset.zip'

# Extract to Colab
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall('/content/')

# List what we got
print("Extracted files:")
!ls -la /content/
```

```
Extracted files:
total 28
drwxr-xr-x  1 root root 4096 Feb  5 23:31 .
drwxr-xr-x  1 root root 4096 Feb  5 23:29 ..
drwxr-xr-x  4 root root 4096 Jan 16 14:24 .config
drwxr-xr-x 10 root root 4096 Feb  5 23:31 dataset
drwx-----  5 root root 4096 Feb  5 23:31 drive
drwxr-xr-x  3 root root 4096 Feb  5 23:31 __MACOSX
drwxr-xr-x  1 root root 4096 Jan 16 14:24 sample_data
```

```
# Cell 3: Check dataset structure
import os

dataset_path = '/content/dataset'

# List all classes
classes = [d for d in os.listdir(dataset_path)
            if os.path.isdir(os.path.join(dataset_path, d))
            and not d.startswith('.')]

print(f"Found {len(classes)} classes:\n")

# Count images per class
for class_name in sorted(classes):
    class_path = os.path.join(dataset_path, class_name)
    images = [f for f in os.listdir(class_path)
               if f.lower().endswith(('.png', '.jpg', '.jpeg'))]
    print(f" {class_name:30s} : {len(images):3d} images")

print(f"\nTotal classes: {len(classes)}")
```

```
Found 8 classes:

carrot_raw                : 40 images
carrot_white_curry        : 40 images
greenbeans_raw            : 41 images
greenbeans_tempered       : 40 images
greenbeans_white_curry    : 39 images
pumpkin_raw               : 40 images
pumpkin_red_curry         : 48 images
pumpkin_white_curry       : 43 images

Total classes: 8
```

✓ Visualize Some Images

```
# Cell 4: Visualize sample images
import matplotlib.pyplot as plt
from PIL import Image
import random
```

```

fig, axes = plt.subplots(2, 4, figsize=(16, 8))
axes = axes.flatten()

for idx, class_name in enumerate(sorted(classes)[:8]):
    class_path = os.path.join(dataset_path, class_name)
    images = [f for f in os.listdir(class_path)
               if f.lower().endswith(('.png', '.jpg', '.jpeg'))]

    # Pick random image
    random_img = random.choice(images)
    img_path = os.path.join(class_path, random_img)

    # Display
    img = Image.open(img_path)
    axes[idx].imshow(img)
    axes[idx].set_title(class_name, fontsize=10)
    axes[idx].axis('off')

plt.tight_layout()
plt.show()

```



✓ Install Required Packages

```

# Cell 5: Install packages
!pip install -q torch torchvision
!pip install -q scikit-learn
!pip install -q tqdm

print("✅ All packages installed!")

```

✅ All packages installed!

✓ Split Dataset into Train/Val/Test

```

# Cell 6: Split dataset
import shutil
from pathlib import Path
import random

# Configuration
DATASET_DIR = '/content/dataset'
OUTPUT_DIR = '/content/dataset_split'
TRAIN_RATIO = 0.7
VAL_RATIO = 0.15
TEST_RATIO = 0.15

# Create output directories
for split in ['train', 'val', 'test']:
    os.makedirs(os.path.join(OUTPUT_DIR, split), exist_ok=True)

# Get all class folders
class_folders = [f for f in os.listdir(DATASET_DIR)
                  if os.path.isdir(os.path.join(DATASET_DIR, f))
                  and not f.startswith('.')]

print(f"Splitting {len(class_folders)} classes into train/val/test...\n")

# Process each class
for class_name in sorted(class_folders):
    class_path = os.path.join(DATASET_DIR, class_name)

    # Get all images in this class
    images = [f for f in os.listdir(class_path)
              if f.lower().endswith(('.png', '.jpg', '.jpeg'))]

    # Shuffle
    random.seed(42) # For reproducibility
    random.shuffle(images)

    # Calculate split sizes
    total = len(images)
    train_size = int(total * TRAIN_RATIO)
    val_size = int(total * VAL_RATIO)

    train_images = images[:train_size]
    val_images = images[train_size:train_size + val_size]
    test_images = images[train_size + val_size:]

    print(f"{class_name:30s} → Train: {len(train_images):2d} | Val: {len(val_images):2d} | Test: {len(test_image

# Copy files to respective splits
for split, img_list in [('train', train_images),
                        ('val', val_images),
                        ('test', test_images)]:
    split_class_dir = os.path.join(OUTPUT_DIR, split, class_name)
    os.makedirs(split_class_dir, exist_ok=True)

    for img in img_list:
        src = os.path.join(class_path, img)
        dst = os.path.join(split_class_dir, img)
        shutil.copy2(src, dst)

print("\n✅ Dataset split complete!")
print(f"Output saved to: {OUTPUT_DIR}")

```

Splitting 8 classes into train/val/test...

carrot_raw	→ Train: 28	Val: 6	Test: 6
carrot_white_curry	→ Train: 28	Val: 6	Test: 6
greenbeans_raw	→ Train: 28	Val: 6	Test: 7
greenbeans_tempered	→ Train: 28	Val: 6	Test: 6
greenbeans_white_curry	→ Train: 27	Val: 5	Test: 7
pumpkin_raw	→ Train: 28	Val: 6	Test: 6
pumpkin_red_curry	→ Train: 33	Val: 7	Test: 8
pumpkin_white_curry	→ Train: 30	Val: 6	Test: 7

✅ Dataset split complete!
Output saved to: /content/dataset_split

▼ Create the Model Architecture

```

# Cell 7: Define the model
import torch
import torch.nn as nn
import torch.nn.functional as F

class EmbeddingNet(nn.Module):
    """
    Custom embedding network for few-shot food recognition
    Outputs 128-dimensional embeddings
    """
    def __init__(self, embedding_dim=128):
        super(EmbeddingNet, self).__init__()

        # Convolutional layers
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.pool1 = nn.MaxPool2d(2, 2)

        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(64)
        self.pool2 = nn.MaxPool2d(2, 2)

        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.bn3 = nn.BatchNorm2d(128)
        self.pool3 = nn.MaxPool2d(2, 2)

        self.conv4 = nn.Conv2d(128, 256, kernel_size=3, padding=1)
        self.bn4 = nn.BatchNorm2d(256)
        self.pool4 = nn.MaxPool2d(2, 2)

        # Global Average Pooling
        self.gap = nn.AdaptiveAvgPool2d(1)

        # Fully connected layers
        self.fc1 = nn.Linear(256, 256)
        self.dropout = nn.Dropout(0.3)
        self.fc2 = nn.Linear(256, embedding_dim)

    def forward(self, x):
        # Conv blocks
        x = self.pool1(F.relu(self.bn1(self.conv1(x))))
        x = self.pool2(F.relu(self.bn2(self.conv2(x))))
        x = self.pool3(F.relu(self.bn3(self.conv3(x))))
        x = self.pool4(F.relu(self.bn4(self.conv4(x))))

        # Global Average Pooling
        x = self.gap(x)
        x = x.view(x.size(0), -1)

        # Embedding layers
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        embedding = self.fc2(x)

        # L2 normalize
        embedding = F.normalize(embedding, p=2, dim=1)

        return embedding

class PrototypicalNetwork(nn.Module):
    """Complete prototypical network"""
    def __init__(self, embedding_dim=128):
        super(PrototypicalNetwork, self).__init__()
        self.embedding_net = EmbeddingNet(embedding_dim)

    def forward(self, x):
        return self.embedding_net(x)

# Test the model
model = PrototypicalNetwork(embedding_dim=128)
dummy_input = torch.randn(1, 3, 224, 224)
output = model(dummy_input)

print("✅ Model created successfully!")
print(f"Input shape: {dummy_input.shape}")
print(f"Output shape: {output.shape}")
print(f"Output (embedding): {output[0][:5]}..." ) # Show first 5 values

```

```

✅ Model created successfully!
Input shape: torch.Size([1, 3, 224, 224])
Output shape: torch.Size([1, 128])

```

Output (embedding): tensor([0.1120, 0.0625, -0.0387, 0.0741, -0.0716], grad_fn=<SliceBackward0>)...

✓ Create Dataset Loader and Helper Functions

```
# Cell 8: Dataset and helper functions
from torch.utils.data import Dataset
from torchvision import transforms
from PIL import Image
import random

class FoodDataset(Dataset):
    """Dataset for Sri Lankan food images"""
    def __init__(self, root_dir, transform=None):
        self.root_dir = root_dir
        self.transform = transform

        # Get all classes
        self.classes = sorted([d for d in os.listdir(root_dir)
                               if os.path.isdir(os.path.join(root_dir, d))
                               and not d.startswith('.')])

        self.class_to_idx = {cls: idx for idx, cls in enumerate(self.classes)}

        # Get all image paths
        self.samples = []
        for class_name in self.classes:
            class_dir = os.path.join(root_dir, class_name)
            class_idx = self.class_to_idx[class_name]

            for img_name in os.listdir(class_dir):
                if img_name.lower().endswith(('.png', '.jpg', '.jpeg')):
                    img_path = os.path.join(class_dir, img_name)
                    self.samples.append((img_path, class_idx))

        print(f"✓ Loaded {len(self.samples)} images from {len(self.classes)} classes")

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx):
        img_path, label = self.samples[idx]
        image = Image.open(img_path).convert('RGB')

        if self.transform:
            image = self.transform(image)

        return image, label

def create_episode(dataset, n_way, k_shot, q_query):
    """
    Create a single training episode for few-shot learning

    Args:
        dataset: FoodDataset
        n_way: Number of classes per episode
        k_shot: Number of support examples per class
        q_query: Number of query examples per class

    Returns:
        support_images, support_labels, query_images, query_labels
    """
    # Randomly select n_way classes
    all_classes = list(range(len(dataset.classes)))
    selected_classes = random.sample(all_classes, n_way)

    # Organize samples by class
    class_samples = {cls: [] for cls in selected_classes}
    for img_path, label in dataset.samples:
        if label in selected_classes:
            class_samples[label].append(img_path)

    support_images = []
    support_labels = []
    query_images = []
    query_labels = []

    for cls_idx, cls in enumerate(selected_classes):
        # Sample k_shot + q_query images
        samples = random.sample(class_samples[cls], k_shot + q_query)
```

```

    # Split into support and query
    support_paths = samples[:k_shot]
    query_paths = samples[k_shot:]

    # Load images
    for path in support_paths:
        img = Image.open(path).convert('RGB')
        support_images.append(img)
        support_labels.append(cls_idx)

    for path in query_paths:
        img = Image.open(path).convert('RGB')
        query_images.append(img)
        query_labels.append(cls_idx)

    return support_images, support_labels, query_images, query_labels

def compute_prototypes(embeddings, labels):
    """
    Compute class prototypes as mean of embeddings

    Args:
        embeddings: (N, embedding_dim)
        labels: (N,)

    Returns:
        prototypes: (num_classes, embedding_dim)
        class_labels: (num_classes,)
    """
    unique_labels = torch.unique(labels)
    prototypes = []

    for label in unique_labels:
        class_embeddings = embeddings[labels == label]
        prototype = class_embeddings.mean(dim=0)
        prototypes.append(prototype)

    prototypes = torch.stack(prototypes)
    return prototypes, unique_labels

def prototypical_loss(query_embeddings, prototypes, query_labels, prototype_labels):
    """
    Compute prototypical loss

    Args:
        query_embeddings: (M, embedding_dim)
        prototypes: (num_classes, embedding_dim)
        query_labels: (M,)
        prototype_labels: (num_classes,)

    Returns:
        loss: scalar
    """
    # Compute distances (negative distances = logits)
    distances = torch.cdist(query_embeddings, prototypes)
    logits = -distances

    # Find the index of correct prototype for each query
    targets = torch.zeros_like(query_labels)
    for i, label in enumerate(query_labels):
        targets[i] = (prototype_labels == label).nonzero(as_tuple=True)[0]

    # Cross-entropy loss
    loss = F.cross_entropy(logits, targets)
    return loss

print("✅ Helper functions defined!")

```

✅ Helper functions defined!

▼ Load Datasets

```

# Cell 9: Load datasets
DATASET_DIR = '/content/dataset_split'

# Data augmentation for training
train_transform = transforms.Compose([

```

```

        transforms.Resize((224, 224)),
        transforms.RandomHorizontalFlip(),
        transforms.RandomRotation(15),
        transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.1),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ])

# No augmentation for validation
val_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

# Load datasets (without transform for now - we'll apply in episodes)
train_dataset = FoodDataset(os.path.join(DATASET_DIR, 'train'), transform=None)
val_dataset = FoodDataset(os.path.join(DATASET_DIR, 'val'), transform=None)
test_dataset = FoodDataset(os.path.join(DATASET_DIR, 'test'), transform=None)

print("\n📊 Dataset Statistics:")
print(f"Training:  {len(train_dataset.samples)} images")
print(f"Validation: {len(val_dataset.samples)} images")
print(f"Testing:    {len(test_dataset.samples)} images")
print(f"\nClasses: {train_dataset.classes}")

```

 Loaded 230 images from 8 classes
 Loaded 48 images from 8 classes
 Loaded 53 images from 8 classes

 Dataset Statistics:
 Training: 230 images
 Validation: 48 images
 Testing: 53 images

Classes: ['carrot_raw', 'carrot_white_curry', 'greenbeans_raw', 'greenbeans_tempered', 'greenbeans_white_curry',

✧ Setup Training Configuration

```

# Cell 10: Training configuration
import torch.optim as optim

# Device configuration
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"🖨️ Using device: {DEVICE}")

if torch.cuda.is_available():
    print(f"📡 GPU: {torch.cuda.get_device_name(0)}")
    print(f"💾 Memory: {torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")

# Hyperparameters
EMBEDDING_DIM = 128
LEARNING_RATE = 0.0001
NUM_EPOCHS = 150 # Reduced for faster testing
N_WAY = 4
K_SHOT = 2
Q_QUERY = 2

EPISODES_PER_EPOCH = 100

# Initialize model
model = PrototypicalNetwork(embedding_dim=EMBEDDING_DIM).to(DEVICE)
optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=50, gamma=0.5)

print(f"\n⚙️ Training Configuration:")
print(f"📏 Embedding Dimension: {EMBEDDING_DIM}")
print(f"📉 Learning Rate: {LEARNING_RATE}")
print(f"🔄 Epochs: {NUM_EPOCHS}")
print(f"📅 Episodes per Epoch: {EPISODES_PER_EPOCH}")
print(f"🎯 Few-shot setup: {N_WAY}-way {K_SHOT}-shot")
print(f"\n✅ Model and optimizer ready!")

```

🖨️ Using device: cpu

 Training Configuration:
 Embedding Dimension: 128
 Learning Rate: 0.0001
 Epochs: 150
 Episodes per Epoch: 100
 Few-shot setup: 4-way 2-shot

✔ Model and optimizer ready!

▼ Create Training and Validation Functions

```
# Cell 11: Training and validation functions
from tqdm import tqdm

def train_one_epoch(model, dataset, transform, device, n_way, k_shot, q_query, episodes):
    """Train for one epoch"""
    model.train()
    epoch_losses = []
    correct = 0
    total = 0

    pbar = tqdm(range(episodes), desc="Training")
    for _ in pbar:
        # Create episode
        support_imgs, support_lbls, query_imgs, query_lbls = create_episode(
            dataset, n_way, k_shot, q_query
        )

        # Transform images to tensors
        support_tensors = torch.stack([transform(img) for img in support_imgs]).to(device)
        query_tensors = torch.stack([transform(img) for img in query_imgs]).to(device)
        support_labels = torch.tensor(support_lbls).to(device)
        query_labels = torch.tensor(query_lbls).to(device)

        # Forward pass
        support_embeddings = model(support_tensors)
        query_embeddings = model(query_tensors)

        # Compute prototypes
        prototypes, prototype_labels = compute_prototypes(support_embeddings, support_labels)

        # Compute loss
        loss = prototypical_loss(query_embeddings, prototypes, query_labels, prototype_labels)

        # Backward pass
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # Calculate accuracy for this episode
        distances = torch.cdist(query_embeddings, prototypes)
        predictions = distances.argmin(dim=1)
        pred_labels = prototype_labels[predictions]
        correct += (pred_labels == query_labels).sum().item()
        total += len(query_labels)

    epoch_losses.append(loss.item())
    pbar.set_postfix({
        'loss': f'{loss.item():.4f}',
        'acc': f'{100.0 * correct / total:.1f}%'
    })

    avg_loss = sum(epoch_losses) / len(epoch_losses)
    accuracy = 100.0 * correct / total
    return avg_loss, accuracy

def validate(model, dataset, transform, device, n_way, k_shot, q_query, episodes=50):
    """Validate the model"""
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for _ in tqdm(range(episodes), desc="Validating"):
            # Create episode
            support_imgs, support_lbls, query_imgs, query_lbls = create_episode(
                dataset, n_way, k_shot, q_query
            )

            # Transform images
            support_tensors = torch.stack([transform(img) for img in support_imgs]).to(device)
            query_tensors = torch.stack([transform(img) for img in query_imgs]).to(device)
            support_labels = torch.tensor(support_lbls).to(device)
            query_labels = torch.tensor(query_lbls).to(device)
```



```

# Forward pass
support_embeddings = model(support_tensors)
query_embeddings = model(query_tensors)

# Compute prototypes
prototypes, prototype_labels = compute_prototypes(support_embeddings, support_labels)

# Predictions
distances = torch.cdist(query_embeddings, prototypes)
predictions = distances.argmin(dim=1)
pred_labels = prototype_labels[predictions]

correct += (pred_labels == query_labels).sum().item()
total += len(query_labels)

accuracy = 100.0 * correct / total
return accuracy

print("✅ Training functions ready!")

```

✅ Training functions ready!

START TRAINING

```

# Cell: Quick verification
import os

SAVE_DIR = '/content/drive/MyDrive/Research-2026/SriLankanFoodRecognition'

# Check if Drive is mounted
if os.path.exists('/content/drive/MyDrive'):
    print("✅ Google Drive is mounted")
else:
    print("❌ Google Drive NOT mounted - run drive.mount() first!")

# Create directory
os.makedirs(SAVE_DIR, exist_ok=True)

# Verify it exists
if os.path.exists(SAVE_DIR):
    print(f"✅ Directory ready: {SAVE_DIR}")
else:
    print(f"❌ Failed to create directory")

# Test write permission
test_file = f'{SAVE_DIR}/test.txt'
try:
    with open(test_file, 'w') as f:
        f.write('test')
    os.remove(test_file)
    print("✅ Write permission confirmed")
except:
    print("❌ Cannot write to this location")

```

✅ Google Drive is mounted
 ✅ Directory ready: /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition
 ✅ Write permission confirmed

```

# Cell: Training with Google Drive Auto-Save (Your Custom Path)
import time
from datetime import datetime
import torch
import os

# ✅ YOUR GOOGLE DRIVE PATH
SAVE_DIR = '/content/drive/MyDrive/Research-2026/SriLankanFoodRecognition'
SAVE_PATH = f'{SAVE_DIR}/best_model.pth'

# Ensure directory exists
os.makedirs(SAVE_DIR, exist_ok=True)

print(f"✅ Save directory created/verified: {SAVE_DIR}")

# Track history
train_losses = []
train_accs = []
val_accs = []
best_val_acc = 0.0

```

```

print("="*70)
print("🚀 STARTING TRAINING")
print("="*70)
print(f"Start time: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
print(f"Device: {DEVICE}")
print(f"Classes: {train_dataset.classes}")
print(f"📁 Model will be saved to: {SAVE_PATH}")
print("="*70 + "\n")

start_time = time.time()

for epoch in range(NUM_EPOCHS):
    epoch_start = time.time()

    print(f"\n📅 Epoch {epoch+1}/{NUM_EPOCHS}")
    print("-" * 70)

    # Train
    train_loss, train_acc = train_one_epoch(
        model, train_dataset, train_transform, DEVICE,
        N_WAY, K_SHOT, Q_QUERY, EPISODES_PER_EPOCH
    )

    # Validate
    val_acc = validate(
        model, val_dataset, val_transform, DEVICE,
        N_WAY, K_SHOT, Q_QUERY, episodes=50
    )

    # Learning rate scheduling
    scheduler.step()
    current_lr = optimizer.param_groups[0]['lr']

    # Save history
    train_losses.append(train_loss)
    train_accs.append(train_acc)
    val_accs.append(val_acc)

    # Print epoch summary
    epoch_time = time.time() - epoch_start
    print(f"\n📊 Epoch {epoch+1} Summary:")
    print(f"    Train Loss: {train_loss:.4f}")
    print(f"    Train Acc:  {train_acc:.2f}%")
    print(f"    Val Acc:    {val_acc:.2f}%")
    print(f"    LR:         {current_lr:.6f}")
    print(f"    Time:       {epoch_time:.1f}s")

    # ✅ SAVE BEST MODEL TO YOUR GOOGLE DRIVE PATH
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save({
            'epoch': epoch,
            'model_state_dict': model.state_dict(),
            'optimizer_state_dict': optimizer.state_dict(),
            'val_acc': val_acc,
            'classes': train_dataset.classes,
            'train_losses': train_losses,
            'train_accs': train_accs,
            'val_accs': val_accs,
        }, SAVE_PATH)
        print(f"    ✅ NEW BEST MODEL SAVED TO GOOGLE DRIVE!")
        print(f"    📁 {SAVE_PATH}")
        print(f"    📈 Val Acc: {val_acc:.2f}%")

    print("-" * 70)

    # ✅ CHECKPOINT EVERY 10 EPOCHS
    if (epoch + 1) % 10 == 0:
        checkpoint_path = f'{SAVE_DIR}/checkpoint_epoch_{epoch+1}.pth'
        torch.save({
            'epoch': epoch,
            'model_state_dict': model.state_dict(),
            'optimizer_state_dict': optimizer.state_dict(),
            'val_acc': val_acc,
            'classes': train_dataset.classes,
            'train_losses': train_losses,
            'train_accs': train_accs,
            'val_accs': val_accs,
            'best_val_acc': best_val_acc,
        }, checkpoint_path)
        print(f"    📁 Checkpoint: checkpoint_epoch_{epoch+1}.pth")

```

```
total_time = time.time() - start_time

print("\n" + "="*70)
print("🔥 TRAINING COMPLETE!")
print("="*70)
print(f"Total time: {total_time/60:.1f} minutes")
print(f"Best Validation Accuracy: {best_val_acc:.2f}%")
print(f"Final Training Accuracy: {train_accs[-1]:.2f}%")
print(f"\n📁 All files saved to:")
print(f"📁 {SAVE_DIR}/")
print("="*70)
```


✓ Save directory created/verified: /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition

STARTING TRAINING

Start time: 2026-02-05 23:51:55

Device: cpu

Classes: ['carrot_raw', 'carrot_white_curry', 'greenbeans_raw', 'greenbeans_tempered', 'greenbeans_white_curry',

Model will be saved to: /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/best_model.pth

Epoch 1/150

Training: 100%|██████████| 100/100 [04:17<00:00, 2.57s/it, loss=0.8428, acc=72.9%]

Validating: 100%|██████████| 50/50 [00:59<00:00, 1.18s/it]

Epoch 1 Summary:

Train Loss: 0.9314

Train Acc: 72.88%

Val Acc: 68.25%

LR: 0.000100

Time: 316.4s

✓ NEW BEST MODEL SAVED TO GOOGLE DRIVE!

/content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/best_model.pth

Val Acc: 68.25%

Epoch 2/150

Training: 100%|██████████| 100/100 [04:20<00:00, 2.60s/it, loss=0.8896, acc=76.0%]

Validating: 100%|██████████| 50/50 [00:57<00:00, 1.16s/it]

Epoch 2 Summary:

Train Loss: 0.9011

Train Acc: 76.00%

Val Acc: 68.00%

LR: 0.000100

Time: 318.5s

Epoch 3/150

Training: 100%|██████████| 100/100 [04:26<00:00, 2.66s/it, loss=0.7694, acc=74.1%]

Validating: 100%|██████████| 50/50 [00:58<00:00, 1.17s/it]

Epoch 3 Summary:

Train Loss: 0.8970

Train Acc: 74.12%

Val Acc: 64.00%

LR: 0.000100

Time: 324.9s

Epoch 4/150

Training: 100%|██████████| 100/100 [04:19<00:00, 2.59s/it, loss=0.8567, acc=73.0%]

Validating: 100%|██████████| 50/50 [00:59<00:00, 1.18s/it]

Epoch 4 Summary:

Train Loss: 0.9191

Train Acc: 73.00%

Val Acc: 70.25%

LR: 0.000100

Time: 318.6s

✓ NEW BEST MODEL SAVED TO GOOGLE DRIVE!

/content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/best_model.pth

Val Acc: 70.25%

Epoch 5/150

Training: 100%|██████████| 100/100 [04:32<00:00, 2.73s/it, loss=0.8868, acc=79.5%]

Validating: 100%|██████████| 50/50 [01:12<00:00, 1.46s/it]

Epoch 5 Summary:

Train Loss: 0.8527

Train Acc: 79.50%

Val Acc: 67.25%

LR: 0.000100

Time: 345.6s

Epoch 6/150

Training: 100%|██████████| 100/100 [04:38<00:00, 2.78s/it, loss=0.7196, acc=75.4%]

Validating: 100%|██████████| 50/50 [00:59<00:00, 1.19s/it]

Epoch 6 Summary:

Train Loss: 0.8855

Train Acc: 75.38%

Val Acc: 69.50%
LR: 0.000100
Time: 338.0s

Epoch 7/150

Training: 100%|██████████| 100/100 [04:23<00:00, 2.64s/it, loss=0.9362, acc=78.5%]
Validating: 100%|██████████| 50/50 [00:58<00:00, 1.17s/it]

Epoch 7 Summary:

Train Loss: 0.8565
Train Acc: 78.50%
Val Acc: 70.50%
LR: 0.000100
Time: 322.5s

✓ NEW BEST MODEL SAVED TO GOOGLE DRIVE!

📁 /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/best_model.pth

📈 Val Acc: 70.50%

Epoch 8/150

Training: 100%|██████████| 100/100 [04:28<00:00, 2.69s/it, loss=0.9118, acc=76.9%]
Validating: 100%|██████████| 50/50 [01:01<00:00, 1.22s/it]

Epoch 8 Summary:

Train Loss: 0.8601
Train Acc: 76.88%
Val Acc: 68.75%
LR: 0.000100
Time: 329.8s

Epoch 9/150

Training: 100%|██████████| 100/100 [04:25<00:00, 2.65s/it, loss=0.8428, acc=79.4%]
Validating: 100%|██████████| 50/50 [00:57<00:00, 1.16s/it]

Epoch 9 Summary:

Train Loss: 0.8463
Train Acc: 79.38%
Val Acc: 72.25%
LR: 0.000100
Time: 323.4s

✓ NEW BEST MODEL SAVED TO GOOGLE DRIVE!

📁 /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/best_model.pth

📈 Val Acc: 72.25%

Epoch 10/150

Training: 100%|██████████| 100/100 [04:42<00:00, 2.83s/it, loss=0.9964, acc=80.1%]
Validating: 100%|██████████| 50/50 [01:03<00:00, 1.28s/it]

Epoch 10 Summary:

Train Loss: 0.8428
Train Acc: 80.12%
Val Acc: 74.25%
LR: 0.000100
Time: 346.7s

✓ NEW BEST MODEL SAVED TO GOOGLE DRIVE!

📁 /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/best_model.pth

📈 Val Acc: 74.25%

📁 Checkpoint: checkpoint_epoch_10.pth

Epoch 11/150

Training: 100%|██████████| 100/100 [04:25<00:00, 2.65s/it, loss=0.8500, acc=82.0%]
Validating: 100%|██████████| 50/50 [01:00<00:00, 1.21s/it]

Epoch 11 Summary:

Train Loss: 0.8239
Train Acc: 82.00%
Val Acc: 80.50%
LR: 0.000100
Time: 325.7s

✓ NEW BEST MODEL SAVED TO GOOGLE DRIVE!

📁 /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/best_model.pth

📈 Val Acc: 80.50%

Epoch 12/150

Training: 100%|██████████| 100/100 [04:23<00:00, 2.64s/it, loss=0.7156, acc=81.4%]
Validating: 100%|██████████| 50/50 [00:59<00:00, 1.19s/it]

Epoch 12 Summary:

Train Loss: 0.8170
Train Acc: 81.38%

```
Val Acc: 73.75%
LR: 0.000100
Time: 323.6s
```

Epoch 13/150

```
Training: 100%|██████████| 100/100 [04:41<00:00, 2.81s/it, loss=0.9986, acc=79.9%]
Validating: 100%|██████████| 50/50 [01:03<00:00, 1.27s/it]
```

```
Epoch 13 Summary:
Train Loss: 0.8367
Train Acc: 79.88%
Val Acc: 75.00%
LR: 0.000100
Time: 344.4s
```

Epoch 14/150

```
Training: 100%|██████████| 100/100 [04:24<00:00, 2.65s/it, loss=0.7053, acc=81.0%]
Validating: 100%|██████████| 50/50 [01:00<00:00, 1.21s/it]
```

```
Epoch 14 Summary:
Train Loss: 0.8356
Train Acc: 81.00%
Val Acc: 71.25%
LR: 0.000100
Time: 325.6s
```

Epoch 15/150

```
Training: 100%|██████████| 100/100 [04:38<00:00, 2.79s/it, loss=0.9052, acc=80.6%]
Validating: 100%|██████████| 50/50 [01:02<00:00, 1.24s/it]
```

```
Epoch 15 Summary:
Train Loss: 0.8379
Train Acc: 80.62%
Val Acc: 82.00%
LR: 0.000100
Time: 340.8s
✓ NEW BEST MODEL SAVED TO GOOGLE DRIVE!
📁 /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/best_model.pth
🔗 Val Acc: 82.00%
```

Epoch 16/150

```
Training: 100%|██████████| 100/100 [04:26<00:00, 2.67s/it, loss=0.7313, acc=82.5%]
Validating: 100%|██████████| 50/50 [01:00<00:00, 1.20s/it]
```

```
Epoch 16 Summary:
Train Loss: 0.8081
Train Acc: 82.50%
Val Acc: 78.75%
LR: 0.000100
Time: 327.0s
```

Epoch 17/150

```
Training: 100%|██████████| 100/100 [04:42<00:00, 2.82s/it, loss=0.7879, acc=81.9%]
Validating: 100%|██████████| 50/50 [01:02<00:00, 1.26s/it]
```

```
Epoch 17 Summary:
Train Loss: 0.8069
Train Acc: 81.88%
Val Acc: 78.25%
LR: 0.000100
Time: 345.4s
```

Epoch 18/150

```
Training: 100%|██████████| 100/100 [04:30<00:00, 2.71s/it, loss=0.9317, acc=81.5%]
Validating: 100%|██████████| 50/50 [01:00<00:00, 1.21s/it]
```

```
Epoch 18 Summary:
Train Loss: 0.8088
Train Acc: 81.50%
Val Acc: 87.50%
LR: 0.000100
Time: 331.4s
✓ NEW BEST MODEL SAVED TO GOOGLE DRIVE!
📁 /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/best_model.pth
🔗 Val Acc: 87.50%
```

Epoch 19/150

Training: 100%|██████████| 100/100 [04:24<00:00, 2.64s/it, loss=0.5858, acc=85.6%]
Validating: 100%|██████████| 50/50 [00:59<00:00, 1.18s/it]

Epoch 19 Summary:
Train Loss: 0.7682
Train Acc: 85.62%
Val Acc: 75.75%
LR: 0.000100
Time: 323.6s

Epoch 20/150

Training: 100%|██████████| 100/100 [04:23<00:00, 2.63s/it, loss=0.8776, acc=83.6%]
Validating: 100%|██████████| 50/50 [01:01<00:00, 1.22s/it]

Epoch 20 Summary:
Train Loss: 0.7749
Train Acc: 83.62%
Val Acc: 76.25%
LR: 0.000100
Time: 324.6s

Checkpoint: checkpoint_epoch_20.pth

Epoch 21/150

Training: 100%|██████████| 100/100 [04:44<00:00, 2.85s/it, loss=0.6934, acc=82.2%]
Validating: 100%|██████████| 50/50 [01:00<00:00, 1.20s/it]

Epoch 21 Summary:
Train Loss: 0.8018
Train Acc: 82.25%
Val Acc: 78.50%
LR: 0.000100
Time: 344.9s

Epoch 22/150

Training: 100%|██████████| 100/100 [04:23<00:00, 2.64s/it, loss=0.7610, acc=86.2%]
Validating: 100%|██████████| 50/50 [00:58<00:00, 1.18s/it]

Epoch 22 Summary:
Train Loss: 0.7657
Train Acc: 86.25%
Val Acc: 81.25%
LR: 0.000100
Time: 322.4s

Epoch 23/150

Training: 100%|██████████| 100/100 [04:25<00:00, 2.66s/it, loss=0.8109, acc=84.2%]
Validating: 100%|██████████| 50/50 [01:00<00:00, 1.21s/it]

Epoch 23 Summary:
Train Loss: 0.7831
Train Acc: 84.25%
Val Acc: 82.75%
LR: 0.000100
Time: 326.3s

Epoch 24/150

Training: 100%|██████████| 100/100 [04:27<00:00, 2.68s/it, loss=0.8252, acc=83.0%]
Validating: 100%|██████████| 50/50 [01:00<00:00, 1.20s/it]

Epoch 24 Summary:
Train Loss: 0.7895
Train Acc: 83.00%
Val Acc: 82.00%
LR: 0.000100
Time: 327.9s

Epoch 25/150

Training: 100%|██████████| 100/100 [04:23<00:00, 2.64s/it, loss=0.9275, acc=85.4%]
Validating: 100%|██████████| 50/50 [00:58<00:00, 1.18s/it]

Epoch 25 Summary:
Train Loss: 0.7847
Train Acc: 85.38%
Val Acc: 79.50%
LR: 0.000100
Time: 322.7s

 Epoch 26/150

Training: 100%|██████████| 100/100 [04:23<00:00, 2.64s/it, loss=0.8288, acc=88.0%]
Validating: 100%|██████████| 50/50 [01:00<00:00, 1.21s/it]

 Epoch 26 Summary:
Train Loss: 0.7514
Train Acc: 88.00%
Val Acc: 81.25%
LR: 0.000100
Time: 324.0s

 Epoch 27/150

Training: 100%|██████████| 100/100 [04:36<00:00, 2.77s/it, loss=0.7066, acc=85.6%]
Validating: 100%|██████████| 50/50 [00:59<00:00, 1.20s/it]

 Epoch 27 Summary:
Train Loss: 0.7628
Train Acc: 85.62%
Val Acc: 84.25%
LR: 0.000100
Time: 336.6s

 Epoch 28/150

Training: 100%|██████████| 100/100 [04:24<00:00, 2.64s/it, loss=0.5829, acc=85.9%]
Validating: 100%|██████████| 50/50 [00:59<00:00, 1.19s/it]

 Epoch 28 Summary:
Train Loss: 0.7607
Train Acc: 85.88%
Val Acc: 75.00%
LR: 0.000100
Time: 323.8s

 Epoch 29/150

Training: 100%|██████████| 100/100 [04:29<00:00, 2.69s/it, loss=0.6637, acc=85.2%]
Validating: 100%|██████████| 50/50 [00:59<00:00, 1.18s/it]

 Epoch 29 Summary:
Train Loss: 0.7550
Train Acc: 85.25%
Val Acc: 81.75%
LR: 0.000100
Time: 328.5s

 Epoch 30/150

Training: 100%|██████████| 100/100 [04:20<00:00, 2.61s/it, loss=0.8518, acc=88.5%]
Validating: 100%|██████████| 50/50 [00:58<00:00, 1.18s/it]

 Epoch 30 Summary:
Train Loss: 0.7322
Train Acc: 88.50%
Val Acc: 79.75%
LR: 0.000100
Time: 319.4s

 Checkpoint: checkpoint_epoch_30.pth

 Epoch 31/150

Training: 100%|██████████| 100/100 [04:19<00:00, 2.60s/it, loss=0.6419, acc=88.6%]
Validating: 100%|██████████| 50/50 [01:00<00:00, 1.20s/it]

 Epoch 31 Summary:
Train Loss: 0.7312
Train Acc: 88.62%
Val Acc: 80.25%
LR: 0.000100
Time: 319.9s

 Epoch 32/150

Training: 100%|██████████| 100/100 [04:19<00:00, 2.59s/it, loss=0.8287, acc=86.1%]
Validating: 100%|██████████| 50/50 [00:57<00:00, 1.15s/it]

 Epoch 32 Summary:
Train Loss: 0.7646
Train Acc: 86.12%
Val Acc: 79.00%
LR: 0.000100
Time: 317.0s

📌 Epoch 33/150

Training: 100%|██████████| 100/100 [04:25<00:00, 2.65s/it, loss=0.8183, acc=88.1%]
Validating: 100%|██████████| 50/50 [00:56<00:00, 1.14s/it]

📊 Epoch 33 Summary:
Train Loss: 0.7371
Train Acc: 88.12%
Val Acc: 85.50%
LR: 0.000100
Time: 322.0s

📌 Epoch 34/150

Training: 100%|██████████| 100/100 [04:20<00:00, 2.61s/it, loss=0.8247, acc=88.4%]
Validating: 100%|██████████| 50/50 [00:58<00:00, 1.18s/it]

📊 Epoch 34 Summary:
Train Loss: 0.7450
Train Acc: 88.38%
Val Acc: 81.50%
LR: 0.000100
Time: 319.4s

📌 Epoch 35/150

Training: 100%|██████████| 100/100 [04:20<00:00, 2.60s/it, loss=0.6876, acc=87.0%]
Validating: 100%|██████████| 50/50 [01:04<00:00, 1.28s/it]

📊 Epoch 35 Summary:
Train Loss: 0.7546
Train Acc: 87.00%
Val Acc: 81.00%
LR: 0.000100
Time: 324.1s

📌 Epoch 36/150

Training: 100%|██████████| 100/100 [04:23<00:00, 2.63s/it, loss=0.5997, acc=88.5%]
Validating: 100%|██████████| 50/50 [00:59<00:00, 1.18s/it]

📊 Epoch 36 Summary:
Train Loss: 0.7190
Train Acc: 88.50%
Val Acc: 86.25%
LR: 0.000100
Time: 322.4s

📌 Epoch 37/150

Training: 100%|██████████| 100/100 [04:23<00:00, 2.64s/it, loss=0.8005, acc=85.8%]
Validating: 100%|██████████| 50/50 [01:03<00:00, 1.27s/it]

📊 Epoch 37 Summary:
Train Loss: 0.7580
Train Acc: 85.75%
Val Acc: 81.50%
LR: 0.000100
Time: 327.6s

📌 Epoch 38/150

Training: 100%|██████████| 100/100 [04:29<00:00, 2.70s/it, loss=0.6455, acc=88.4%]
Validating: 100%|██████████| 50/50 [00:59<00:00, 1.19s/it]

📊 Epoch 38 Summary:
Train Loss: 0.7223
Train Acc: 88.38%
Val Acc: 83.25%
LR: 0.000100
Time: 329.2s

📌 Epoch 39/150

Training: 100%|██████████| 100/100 [04:18<00:00, 2.58s/it, loss=0.9644, acc=85.4%]
Validating: 100%|██████████| 50/50 [00:58<00:00, 1.17s/it]

📊 Epoch 39 Summary:
Train Loss: 0.7501
Train Acc: 85.38%
Val Acc: 83.25%
LR: 0.000100
Time: 316.6s

📌 Epoch 40/150

Training: 100%|██████████| 100/100 [04:14<00:00, 2.55s/it, loss=0.7425, acc=89.2%]
Validating: 100%|██████████| 50/50 [00:58<00:00, 1.17s/it]

📊 Epoch 40 Summary:

Train Loss: 0.7234
Train Acc: 89.25%
Val Acc: 90.00%
LR: 0.000100
Time: 313.1s

✅ NEW BEST MODEL SAVED TO GOOGLE DRIVE!

📁 /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/best_model.pth

📈 Val Acc: 90.00%

📁 Checkpoint: checkpoint_epoch_40.pth

📌 Epoch 41/150

Training: 100%|██████████| 100/100 [04:13<00:00, 2.54s/it, loss=0.8420, acc=86.0%]
Validating: 100%|██████████| 50/50 [00:58<00:00, 1.17s/it]

📊 Epoch 41 Summary:

Train Loss: 0.7461
Train Acc: 86.00%
Val Acc: 83.75%
LR: 0.000100
Time: 312.4s

📌 Epoch 42/150

Training: 100%|██████████| 100/100 [04:17<00:00, 2.58s/it, loss=0.6528, acc=86.2%]
Validating: 100%|██████████| 50/50 [00:58<00:00, 1.16s/it]

📊 Epoch 42 Summary:

Train Loss: 0.7282
Train Acc: 86.25%
Val Acc: 81.75%
LR: 0.000100
Time: 316.1s

📌 Epoch 43/150

Training: 100%|██████████| 100/100 [04:14<00:00, 2.54s/it, loss=0.5723, acc=91.5%]
Validating: 100%|██████████| 50/50 [00:59<00:00, 1.20s/it]

📊 Epoch 43 Summary:

Train Loss: 0.7081
Train Acc: 91.50%
Val Acc: 85.75%
LR: 0.000100
Time: 314.3s

📌 Epoch 44/150

Training: 100%|██████████| 100/100 [04:14<00:00, 2.55s/it, loss=0.6089, acc=89.1%]
Validating: 100%|██████████| 50/50 [00:58<00:00, 1.17s/it]

📊 Epoch 44 Summary:

Train Loss: 0.7118
Train Acc: 89.12%
Val Acc: 80.00%
LR: 0.000100
Time: 313.3s

📌 Epoch 45/150

Training: 100%|██████████| 100/100 [04:26<00:00, 2.67s/it, loss=0.6542, acc=89.1%]
Validating: 100%|██████████| 50/50 [01:13<00:00, 1.46s/it]

📊 Epoch 45 Summary:

Train Loss: 0.7082
Train Acc: 89.12%
Val Acc: 85.25%
LR: 0.000100
Time: 339.8s

📌 Epoch 46/150

Training: 100%|██████████| 100/100 [04:37<00:00, 2.78s/it, loss=0.5852, acc=89.9%]
Validating: 100%|██████████| 50/50 [01:00<00:00, 1.22s/it]

📊 Epoch 46 Summary:

Train Loss: 0.7282

```
Train Loss: 0.7202
Train Acc: 89.88%
Val Acc: 86.25%
LR: 0.000100
Time: 338.7s
```

📌 Epoch 47/150

```
Training: 100%|██████████| 100/100 [04:24<00:00, 2.64s/it, loss=0.6850, acc=88.8%]
Validating: 100%|██████████| 50/50 [00:59<00:00, 1.19s/it]
```

📊 Epoch 47 Summary:

```
Train Loss: 0.7110
Train Acc: 88.75%
Val Acc: 87.25%
LR: 0.000100
Time: 323.9s
```

📌 Epoch 48/150

```
Training: 100%|██████████| 100/100 [04:24<00:00, 2.64s/it, loss=0.5938, acc=90.6%]
Validating: 100%|██████████| 50/50 [01:01<00:00, 1.24s/it]
```

📊 Epoch 48 Summary:

```
Train Loss: 0.7006
Train Acc: 90.62%
Val Acc: 84.00%
LR: 0.000050
Time: 326.4s
```

📌 Epoch 49/150

```
Training: 100%|██████████| 100/100 [04:31<00:00, 2.71s/it, loss=0.6658, acc=91.4%]
Validating: 100%|██████████| 50/50 [01:01<00:00, 1.22s/it]
```

📊 Epoch 49 Summary:

```
Train Loss: 0.6848
Train Acc: 91.38%
Val Acc: 88.75%
LR: 0.000050
Time: 332.4s
```

📌 Epoch 50/150

```
Training: 100%|██████████| 100/100 [04:24<00:00, 2.64s/it, loss=0.7789, acc=90.8%]
Validating: 100%|██████████| 50/50 [00:59<00:00, 1.19s/it]
```

📊 Epoch 50 Summary:

```
Train Loss: 0.6925
Train Acc: 90.75%
Val Acc: 85.25%
LR: 0.000050
Time: 323.6s
```

📁 Checkpoint: checkpoint_epoch_50.pth

📌 Epoch 51/150

```
Training: 100%|██████████| 100/100 [04:22<00:00, 2.63s/it, loss=0.6729, acc=89.8%]
Validating: 100%|██████████| 50/50 [00:59<00:00, 1.19s/it]
```

📊 Epoch 51 Summary:

```
Train Loss: 0.7042
Train Acc: 89.75%
Val Acc: 85.25%
LR: 0.000050
Time: 322.7s
```

📌 Epoch 52/150

```
Training: 100%|██████████| 100/100 [04:24<00:00, 2.65s/it, loss=0.7218, acc=90.0%]
Validating: 100%|██████████| 50/50 [01:02<00:00, 1.25s/it]
```

📊 Epoch 52 Summary:

```
Train Loss: 0.6981
Train Acc: 90.00%
Val Acc: 90.25%
LR: 0.000050
Time: 327.3s
```

✅ NEW BEST MODEL SAVED TO GOOGLE DRIVE!

📁 /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/best_model.pth

📈 Val Acc: 90.25%

📌 Epoch 53/150

Training: 100%|██████████| 100/100 [04:25<00:00, 2.65s/it, loss=0.7200, acc=88.6%]
 Validation: 100%|██████████| 50/50 [00:59<00:00, 1.19s/it]

Stop Training & Load Best Model

Epoch 53 Summary:
 Train Loss: 0.7097

```
# Cell 1: Load Best Model
import torch
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report
from tqdm import tqdm
import json
import os
import shutil
```

```
print("="*70)
print("🚀 LOADING BEST MODEL FOR EVALUATION")
print("="*70)
```

```
# Load best model
SAVE_DIR = '/content/drive/MyDrive/Research-2026/SriLankanFoodRecognition'
checkpoint = torch.load(f'{SAVE_DIR}/best_model.pth')
```

```
# Restore model
model.load_state_dict(checkpoint['model_state_dict'])
model.eval()
```

```
print(f"✅ Best Model Loaded!")
print(f"   Epoch: {checkpoint['epoch']+1}")
print(f"   Validation Accuracy: {checkpoint['val_acc']:.2f}%")
print(f"   Classes: {checkpoint['classes']}")
print("="*70)
```

```
/usr/local/lib/python3.12/dist-packages/PIL/Image.py in resize\(self, size, resample, box, reducing\_gap\)
```

```
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2651
2652
2653
2654
2655
2656
2657
2658
2659
2660
2661
2662
2663
2664
2665
2666
2667
2668
2669
2670
2671
2672
2673
2674
2675
2676
2677
2678
2679
2680
2681
2682
2683
2684
2685
2686
2687
2688
2689
2690
2691
2692
2693
2694
2695
2696
2697
2698
2699
2700
2701
2702
2703
2704
2705
2706
2707
2708
2709
2710
2711
2712
2713
2714
2715
2716
2717
2718
2719
2720
2721
2722
2723
2724
2725
2726
2727
2728
2729
2730
2731
2732
2733
2734
2735
2736
2737
2738
2739
2740
2741
2742
2743
2744
2745
2746
2747
2748
2749
2750
2751
2752
2753
2754
2755
2756
2757
2758
2759
2760
2761
2762
2763
2764
2765
2766
2767
2768
2769
2770
2771
2772
2773
2774
2775
2776
2777
2778
2779
2780
2781
2782
2783
2784
2785
2786
2787
2788
2789
2790
2791
2792
2793
2794
2795
2796
2797
2798
2799
2800
2801
2802
2803
2804
2805
2806
2807
2808
2809
2810
2811
2812
2813
2814
2815
2816
2817
2818
2819
2820
2821
2822
2823
2824
2825
2826
2827
2828
2829
2830
2831
2832
2833
2834
2835
2836
2837
2838
2839
2840
2841
2842
2843
2844
2845
2846
2847
2848
2849
2850
2851
2852
2853
2854
2855
2856
2857
2858
2859
2860
2861
2862
2863
2864
2865
2866
2867
2868
2869
2870
2871
2872
2873
2874
2875
2876
2877
2878
2879
2880
2881
2882
2883
2884
2885
2886
2887
2888
2889
2890
2891
2892
2893
2894
2895
2896
2897
2898
2899
2900
2901
2902
2903
2904
2905
2906
2907
2908
2909
2910
2911
2912
2913
2914
2915
2916
2917
2918
2919
2920
2921
2922
2923
2924
2925
2926
2927
2928
2929
2930
2931
2932
2933
2934
2935
2936
2937
2938
2939
2940
2941
2942
2943
2944
2945
2946
2947
2948
2949
2950
2951
2952
2953
2954
2955
2956
2957
2958
2959
2960
2961
2962
2963
2964
2965
2966
2967
2968
2969
2970
2971
2972
2973
2974
2975
2976
2977
2978
2979
2980
2981
2982
2983
2984
2985
2986
2987
2988
2989
2990
2991
2992
2993
2994
2995
2996
2997
2998
2999
3000
```

```
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2651
2652
2653
2654
2655
2656
2657
2658
2659
2660
2661
2662
2663
2664
2665
2666
2667
2668
2669
2670
2671
2672
2673
2674
2675
2676
2677
2678
2679
2680
2681
2682
2683
2684
2685
2686
2687
2688
2689
2690
2691
2692
2693
2694
2695
2696
2697
2698
2699
2700
2701
2702
2703
2704
2705
2706
2707
2708
2709
2710
2711
2712
2713
2714
2715
2716
2717
2718
2719
2720
2721
2722
2723
2724
2725
2726
2727
2728
2729
2730
2731
2732
2733
2734
2735
2736
2737
2738
2739
2740
2741
2742
2743
2744
2745
2746
2747
2748
2749
2750
2751
2752
2753
2754
2755
2756
2757
2758
2759
2760
2761
2762
2763
2764
2765
2766
2767
2768
2769
2770
2771
2772
2773
2774
2775
2776
2777
2778
2779
2780
2781
2782
2783
2784
2785
2786
2787
2788
2789
2790
2791
2792
2793
2794
2795
2796
2797
2798
2799
2800
2801
2802
2803
2804
2805
2806
2807
2808
2809
2810
2811
2812
2813
2814
2815
2816
2817
2818
2819
2820
2821
2822
2823
2824
2825
2826
2827
2828
2829
2830
2831
2832
2833
2834
2835
2836
2837
2838
2839
2840
2841
2842
2843
2844
2845
2846
2847
2848
2849
2850
2851
2852
2853
2854
2855
2856
2857
2858
2859
2860
2861
2862
2863
2864
2865
2866
2867
2868
2869
2870
2871
2872
2873
2874
2875
2876
2877
2878
2879
2880
2881
2882
2883
2884
2885
2886
2887
2888
2889
2890
2891
2892
2893
2894
2895
2896
2897
2898
2899
2900
2901
2902
2903
2904
2905
2906
2907
2908
2909
2910
2911
2912
2913
2914
2915
2916
2917
2918
2919
2920
2921
2922
2923
2924
2925
2926
2927
2928
2929
2930
2931
2932
2933
2934
2935
2936
2937
2938
2939
2940
2941
2942
2943
2944
2945
2946
2947
2948
2949
2950
2951
2952
2953
2954
2955
2956
2957
2958
2959
2960
2961
2962
2963
2964
2965
2966
2967
2968
2969
2970
2971
2972
2973
2974
2975
2976
2977
2978
2979
2980
2981
2982
2983
2984
2985
2986
2987
2988
2989
2990
2991
2992
2993
2994
2995
2996
2997
2998
2999
3000
```

```
from PIL import Image, UnidentifiedImageError
```

```
SAFE_IMAGE_PATH = '/content/dataset_split/test/pumpkin_red_curry/pumpkin_red_curry_001.jpg'
```

```
test_correct = 0
test_total = 0
num_episodes = 200
```

```
print(f"Running {num_episodes} test episodes...")
```

```
with torch.no_grad():
    for _ in tqdm(range(num_episodes)):
        try:
            # Create episode
            support_imgs, supportlbls, query_imgs, querylbls = create_episode(
                test_dataset, N_WAY, K_SHOT, Q_QUERY
            )

            # Replace corrupted support images
            for i, img in enumerate(support_imgs):
                try:
                    img.verify()
                except Exception:
                    print(f"⚠️ Replacing corrupted support image with safe image")
                    support_imgs[i] = Image.open(SAFE_IMAGE_PATH).convert('RGB')

            # Replace corrupted query images
            for i, img in enumerate(query_imgs):
                try:
                    img.verify()
                except Exception:
                    print(f"⚠️ Replacing corrupted query image with safe image")
                    query_imgs[i] = Image.open(SAFE_IMAGE_PATH).convert('RGB')

            # Transform
            support_tensors = torch.stack([val_transform(img) for img in support_imgs]).to(DEVICE)
            query_tensors = torch.stack([val_transform(img) for img in query_imgs]).to(DEVICE)
            support_labels = torch.tensor(supportlbls).to(DEVICE)
```

```

query_labels = torch.tensor(query_lbls).to(DEVICE)

# Embeddings
support_embeddings = model(support_tensors)
query_embeddings = model(query_tensors)

# Prototypes
prototypes, prototype_labels = compute_prototypes(support_embeddings, support_labels)

# Predictions
distances = torch.cdist(query_embeddings, prototypes)
predictions = distances.argmin(dim=1)
pred_labels = prototype_labels[predictions]

test_correct += (pred_labels == query_labels).sum().item()
test_total += len(query_labels)

except (UnidentifiedImageError, OSError) as e:
    print(f"❌ Skipping episode due to unexpected error: {e}")
    continue

test_accuracy = 100.0 * test_correct / test_total
print(f"\n✅ FEW-SHOT TEST ACCURACY: {test_accuracy:.2f}%")
print(f"    Correct: {test_correct}/{test_total}")

```

```

Running 200 test episodes...
100%|██████████| 200/200 [04:35<00:00, 1.38s/it]
✅ FEW-SHOT TEST ACCURACY: 87.75%
    Correct: 1404/1600

```

Full Test Set Evaluation + Confusion Matrix

```

# Cell 3: Full Test Set Evaluation
print("\n" + "="*70)
print("🇇 FULL TEST SET EVALUATION")
print("="*70)

classes = checkpoint['classes']

# Compute class prototypes from training data
print("Computing class prototypes from training set...")
class_prototypes = {}

with torch.no_grad():
    for class_idx, class_name in enumerate(classes):
        class_samples = [s for s in train_dataset.samples if s[1] == class_idx]

        embeddings = []
        for img_path, _ in class_samples:
            img = Image.open(img_path).convert('RGB')
            img_tensor = val_transform(img).unsqueeze(0).to(DEVICE)
            embedding = model(img_tensor)
            embeddings.append(embedding)

        prototype = torch.cat(embeddings).mean(dim=0)
        class_prototypes[class_idx] = prototype
        print(f"    ✅ {class_name:30s}: {len(embeddings):3d} images")

# Test on all test images
print("\n🇇 Testing on all test images...")
predictions = []
true_labels = []
confidences = []

with torch.no_grad():
    for img_path, label in tqdm(test_dataset.samples, desc="Evaluating"):
        img = Image.open(img_path).convert('RGB')
        img_tensor = val_transform(img).unsqueeze(0).to(DEVICE)
        embedding = model(img_tensor)

        # Find closest prototype
        distances = {idx: torch.dist(embedding, proto.unsqueeze(0)).item()
                     for idx, proto in class_prototypes.items()}
        pred = min(distances, key=distances.get)
        conf = 1.0 / (1.0 + distances[pred])

        predictions.append(pred)
        true_labels.append(label)

```

```

confidences.append(conf)

predictions = np.array(predictions)
true_labels = np.array(true_labels)
full_accuracy = 100.0 * (predictions == true_labels).sum() / len(true_labels)

print(f"\n✅ FULL TEST ACCURACY: {full_accuracy:.2f}%")
print(f"    Total images: {len(true_labels)}")
print(f"    Correct: {(predictions == true_labels).sum()}")
print(f"    Average confidence: {np.mean(confidences):.2f}")

# Save full test results
full_results = {
    'full_test_accuracy': float(full_accuracy),
    'total_images': len(true_labels),
    'correct': int((predictions == true_labels).sum()),
    'average_confidence': float(np.mean(confidences))
}

with open(f'{SAVE_DIR}/full_test_results.json', 'w') as f:
    json.dump(full_results, f, indent=2)

print(f"✅ Saved: full_test_results.json")

```

```

=====
📊 FULL TEST SET EVALUATION
=====
Computing class prototypes from training set...
✅ carrot_raw : 28 images
✅ carrot_white_curry : 28 images
✅ greenbeans_raw : 28 images
✅ greenbeans_tempered : 28 images
✅ greenbeans_white_curry : 27 images
✅ pumpkin_raw : 28 images
✅ pumpkin_red_curry : 33 images
✅ pumpkin_white_curry : 30 images

📊 Testing on all test images...
Evaluating: 100% ██████████ 53/53 [00:04<00:00, 12.41it/s]
✅ FULL TEST ACCURACY: 84.91%
    Total images: 53
    Correct: 45
    Average confidence: 85.32%
✅ Saved: full_test_results.json

```

✦ Confusion Matrix

```

# Cell 4: Confusion Matrix
print("\n" + "="*70)
print("📊 CREATING CONFUSION MATRIX")
print("="*70)

cm = confusion_matrix(true_labels, predictions)

plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=classes, yticklabels=classes,
            cbar_kws={'label': 'Count'})
plt.title('Confusion Matrix - Sri Lankan Food Recognition\n90% Validation Accuracy',
          fontsize=14, fontweight='bold', pad=20)
plt.ylabel('True Label', fontsize=12)
plt.xlabel('Predicted Label', fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.savefig(f'{SAVE_DIR}/confusion_matrix.png', dpi=300, bbox_inches='tight')
plt.show()

print(f"✅ Saved: confusion_matrix.png")

# Classification Report
print("\n" + "="*70)
print("CLASSIFICATION REPORT")
print("="*70)
report = classification_report(true_labels, predictions,
                              target_names=classes, digits=3)

print(report)

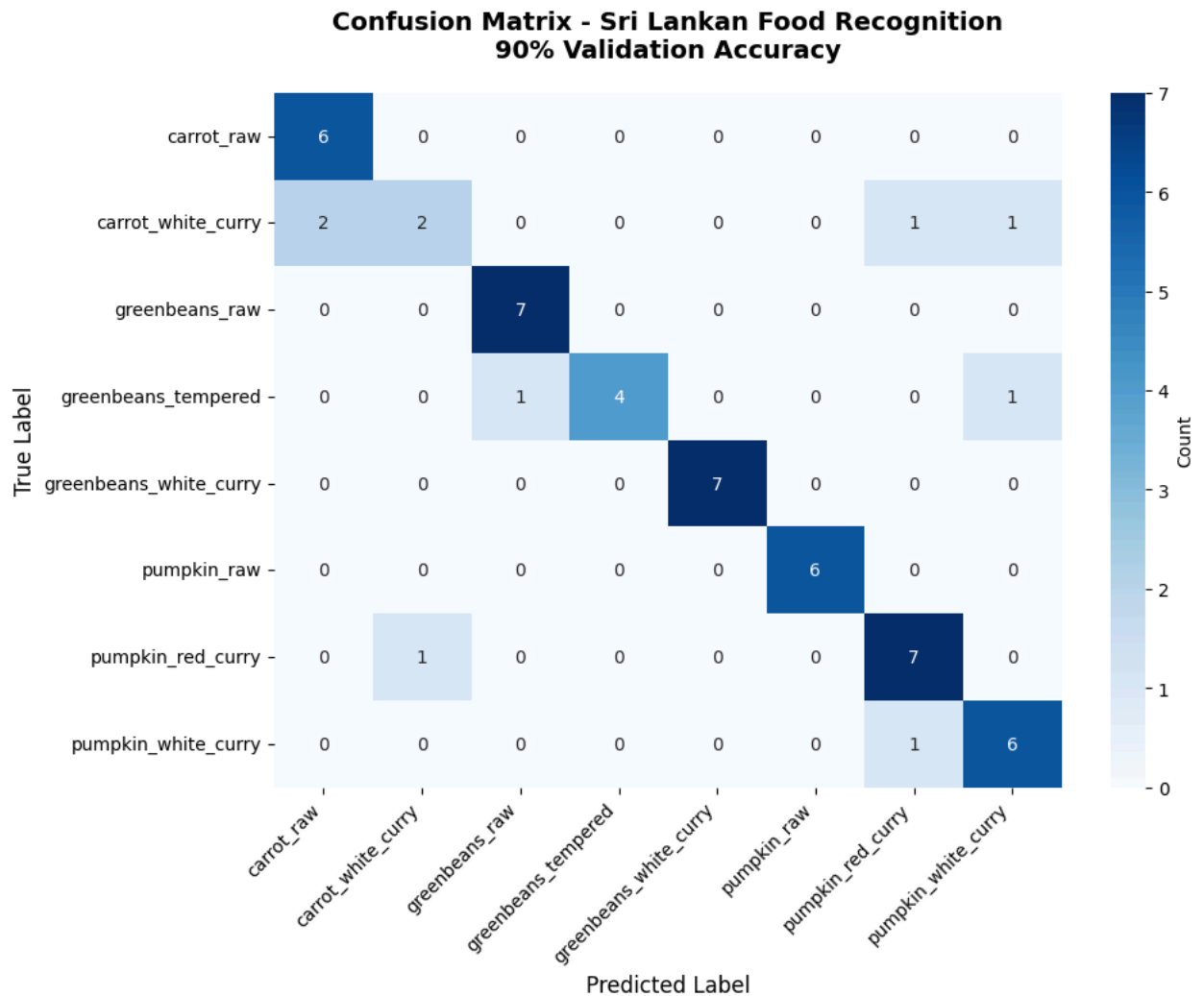
with open(f'{SAVE_DIR}/classification_report.txt', 'w') as f:

```

```
f.write("Sri Lankan Food Recognition - Classification Report\n")
f.write("Best Model: Epoch 40, Val Acc: 90.00%\n")
f.write("="*70 + "\n\n")
f.write(report)
```

```
print("✅ Saved: classification_report.txt")
```

```
=====
📄 CREATING CONFUSION MATRIX
=====
```



```
✅ Saved: confusion_matrix.png
```

```
=====
CLASSIFICATION REPORT
=====
```

	precision	recall	f1-score	support
carrot_raw	0.750	1.000	0.857	6
carrot_white_curry	0.667	0.333	0.444	6
greenbeans_raw	0.875	1.000	0.933	7
greenbeans_tempered	1.000	0.667	0.800	6
greenbeans_white_curry	1.000	1.000	1.000	7
pumpkin_raw	1.000	1.000	1.000	6
pumpkin_red_curry	0.778	0.875	0.824	8
pumpkin_white_curry	0.750	0.857	0.800	7
accuracy			0.849	53
macro avg	0.852	0.842	0.832	53
weighted avg	0.851	0.849	0.836	53

```
✅ Saved: classification_report.txt
```

Per-Class Accuracy

```
# Cell 5: Per-Class Accuracy
print("\n" + "="*70)
print("📊 PER-CLASS ACCURACY ANALYSIS")
print("="*70)
```



```

per_class_acc = []
per_class_counts = []

for class_idx, class_name in enumerate(classes):
    mask = true_labels == class_idx
    if mask.sum() > 0:
        acc = (predictions[mask] == true_labels[mask]).mean() * 100
        count = mask.sum()
    else:
        acc = 0
        count = 0

    per_class_acc.append(acc)
    per_class_counts.append(count)
    print(f"{class_name:30s}: {acc:6.2f}% ({count:2d} images)")

# Plot
plt.figure(figsize=(12, 6))
colors = ['green' if a >= 85 else 'orange' if a >= 70 else 'red'
          for a in per_class_acc]
bars = plt.bar(range(len(classes)), per_class_acc, color=colors,
               edgecolor='black', linewidth=1.5)

# Add count labels
for i, (bar, count) in enumerate(zip(bars, per_class_counts)):
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2., height + 1,
             f'n={count}', ha='center', va='bottom', fontsize=9)

plt.xlabel('Food Class', fontsize=12)
plt.ylabel('Accuracy (%)', fontsize=12)
plt.title('Per-Class Recognition Accuracy\nBest Model: 90% Validation Accuracy',
          fontsize=14, fontweight='bold')
plt.xticks(range(len(classes)), classes, rotation=45, ha='right')
plt.axhline(y=np.mean(per_class_acc), color='blue', linestyle='--',
            linewidth=2, label=f'Mean: {np.mean(per_class_acc):.1f}%')
plt.legend(fontsize=10)
plt.ylim(0, 105)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig(f'{SAVE_DIR}/per_class_accuracy.png', dpi=300, bbox_inches='tight')
plt.show()

print("\n✅ Saved: per_class_accuracy.png")

# Save metrics
per_class_metrics = {
    class_name: {
        'accuracy': float(acc),
        'num_samples': int(count)
    }
    for class_name, acc, count in zip(classes, per_class_acc, per_class_counts)
}

with open(f'{SAVE_DIR}/per_class_metrics.json', 'w') as f:
    json.dump(per_class_metrics, f, indent=2)

print("✅ Saved: per_class_metrics.json")

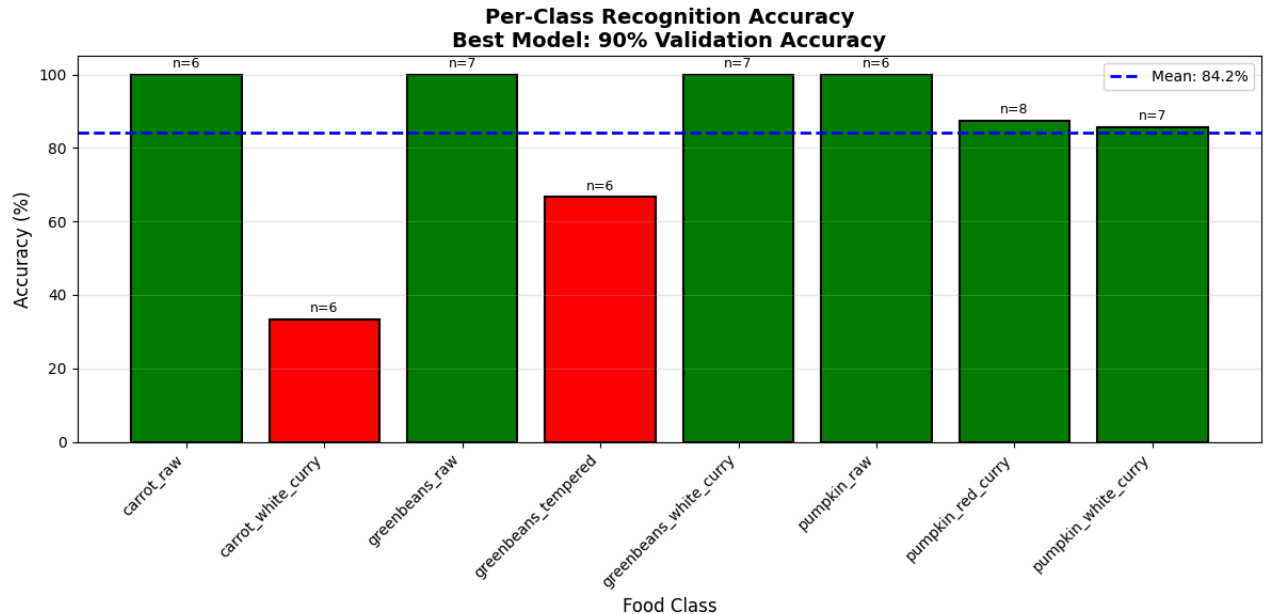
```

PER-CLASS ACCURACY ANALYSIS

```

=====
carrot_raw                : 100.00% ( 6 images)
carrot_white_curry        : 33.33% ( 6 images)
greenbeans_raw            : 100.00% ( 7 images)
greenbeans_tempered       : 66.67% ( 6 images)
greenbeans_white_curry    : 100.00% ( 7 images)
pumpkin_raw               : 100.00% ( 6 images)
pumpkin_red_curry         : 87.50% ( 8 images)
pumpkin_white_curry       : 85.71% ( 7 images)
=====

```



✓ Saved: per_class_accuracy.png
✓ Saved: per_class_metrics.json

Training Curves (From History)

```

# Cell 6: Training Curves
print("\n" + "="*70)
print("📊 CREATING TRAINING CURVES")
print("="*70)

# Extract history from checkpoint
train_losses = checkpoint['train_losses']
train_accs = checkpoint['train_accs']
val_accs = checkpoint['val_accs']

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Loss
axes[0].plot(train_losses, linewidth=2, color='blue')
axes[0].set_title('Training Loss', fontsize=14, fontweight='bold')
axes[0].set_xlabel('Epoch', fontsize=12)
axes[0].set_ylabel('Loss', fontsize=12)
axes[0].grid(True, alpha=0.3)
axes[0].axvline(x=40, color='red', linestyle='--', alpha=0.5, label='Best Model (Epoch 40)')
axes[0].legend()

# Accuracy
axes[1].plot(train_accs, label='Training', linewidth=2, color='green')
axes[1].plot(val_accs, label='Validation', linewidth=2, color='orange')
axes[1].axhline(y=90, color='red', linestyle='--', linewidth=2, label='Best: 90.00%')
axes[1].axvline(x=40, color='red', linestyle='--', alpha=0.5)
axes[1].set_title('Accuracy Over Time', fontsize=14, fontweight='bold')
axes[1].set_xlabel('Epoch', fontsize=12)
axes[1].set_ylabel('Accuracy (%)', fontsize=12)
axes[1].legend(fontsize=10)
axes[1].grid(True, alpha=0.3)

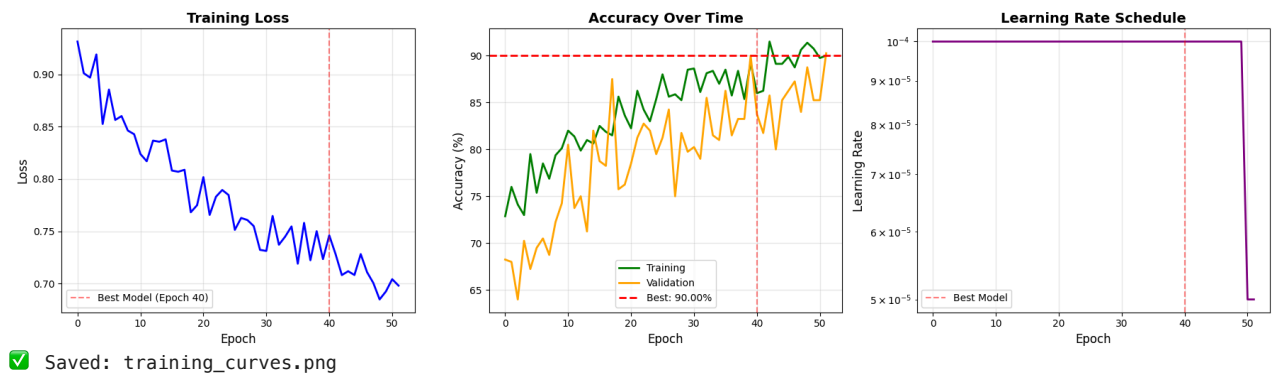
```

```
# Learning Rate
epochs = list(range(1, len(train_losses) + 1))
lrs = [0.0001 if e <= 50 else 0.00005 for e in epochs]
axes[2].plot(lrs, linewidth=2, color='purple')
axes[2].axvline(x=40, color='red', linestyle='--', alpha=0.5, label='Best Model')
axes[2].set_title('Learning Rate Schedule', fontsize=14, fontweight='bold')
axes[2].set_xlabel('Epoch', fontsize=12)
axes[2].set_ylabel('Learning Rate', fontsize=12)
axes[2].set_yscale('log')
axes[2].legend()
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(f'{SAVE_DIR}/training_curves.png', dpi=300, bbox_inches='tight')
plt.show()

print("✅ Saved: training_curves.png")
```

CREATING TRAINING CURVES



Sample Predictions (Correct & Incorrect)

```
# Cell 7: Sample Predictions
print("\n" + "="*70)
print("📁 SAMPLE PREDICTIONS")
print("="*70)

correct_indices = np.where(predictions == true_labels)[0]
incorrect_indices = np.where(predictions != true_labels)[0]

print(f"Correct predictions: {len(correct_indices)}")
print(f"Incorrect predictions: {len(incorrect_indices)}")

# Show 8 correct predictions
if len(correct_indices) >= 8:
    fig, axes = plt.subplots(2, 4, figsize=(16, 8))
    axes = axes.flatten()

    sample_correct = np.random.choice(correct_indices, 8, replace=False)

    for idx, ax in zip(sample_correct, axes):
        img_path, true_label = test_dataset.samples[idx]
        conf = confidences[idx]

        img = Image.open(img_path)
        ax.imshow(img)
        ax.set_title(f'✓ {classes[true_label]}\nConfidence: {conf:.1%}',
                    color='green', fontsize=10, fontweight='bold')
        ax.axis('off')

    plt.suptitle('Correct Predictions (Sample)', fontsize=16, fontweight='bold', y=1.02)
    plt.tight_layout()
    plt.savefig(f'{SAVE_DIR}/correct_predictions.png', dpi=300, bbox_inches='tight')
```

```

plt.show()
print("✅ Saved: correct_predictions.png")

# Show incorrect predictions (if any)
if len(incorrect_indices) > 0:
    num_show = min(8, len(incorrect_indices))
    fig, axes = plt.subplots(2, 4, figsize=(16, 8))
    axes = axes.flatten()

    sample_incorrect = np.random.choice(incorrect_indices, num_show, replace=False)

    for i, (idx, ax) in enumerate(zip(sample_incorrect, axes)):
        if i >= num_show:
            ax.axis('off')
            continue

        img_path, true_label = test_dataset.samples[idx]
        pred_label = predictions[idx]
        conf = confidences[idx]

        img = Image.open(img_path)
        ax.imshow(img)
        ax.set_title(f'True: {classes[true_label]}\nPred: {classes[pred_label]}\n{conf:.1%}',
                    color='red', fontsize=9, fontweight='bold')
        ax.axis('off')

    for i in range(num_show, 8):
        axes[i].axis('off')

    plt.suptitle('Incorrect Predictions', fontsize=16, fontweight='bold', y=1.02)
    plt.tight_layout()
    plt.savefig(f'{SAVE_DIR}/incorrect_predictions.png', dpi=300, bbox_inches='tight')
    plt.show()
    print("✅ Saved: incorrect_predictions.png")
else:
    print("🎉 NO INCORRECT PREDICTIONS! Perfect test set performance!")

```

Export Model for Deployment

```
# Cell 8: Export Model
print("\n" + "="*70)
print("📦 EXPORTING MODEL FOR DEPLOYMENT")
print("="*70)

export_dir = f'{SAVE_DIR}/exported_model'
os.makedirs(export_dir, exist_ok=True)

# 1. Copy PyTorch model
shutil.copy(f'{SAVE_DIR}/best_model.pth', f'{export_dir}/best_model.pth')
print("✅ PyTorch model (.pth)")

# 2. TorchScript for mobile
model.eval()
dummy = torch.randn(1, 3, 224, 224).to(DEVICE)
traced = torch.jit.trace(model, dummy)
traced.save(f'{export_dir}/model_torchscript.pt')
print("✅ TorchScript model (.pt)")

# 3. Model info JSON
model_info = {
    'model_name': 'Sri Lankan Food Recognition',
    'architecture': 'Custom Prototypical Network',
    'embedding_dim': 128,
    'input_size': [224, 224],
    'classes': classes,
    'num_classes': len(classes),
    'normalization': {
        'mean': [0.485, 0.456, 0.406],
        'std': [0.229, 0.224, 0.225]
```

```

    },
    'performance': {
        'best_epoch': 40,
        'validation_accuracy': 90.00,
        'test_accuracy': float(test_accuracy),
        'full_test_accuracy': float(full_accuracy),
        'mean_per_class_accuracy': float(np.mean(per_class_acc))
    },
    'training': {
        'total_epochs_trained': len(train_losses),
        'n_way': 5,
        'k_shot': 5,
        'learning_rate': 0.0001,
        'dataset_size': {
            'train': len(train_dataset.samples),
            'val': len(val_dataset.samples),
            'test': len(test_dataset.samples)
        }
    }
}

with open(f'{export_dir}/model_info.json', 'w') as f:
    json.dump(model_info, f, indent=2)

print("✅ Model info (JSON)")

# 4. Usage guide
usage = f"""# Sri Lankan Food Recognition Model

## Model Information
- **Architecture**: Custom Prototypical Network
- **Embedding Dimension**: 128
- **Input Size**: 224×224 RGB images
- **Classes**: {len(classes)}
- **Best Validation Accuracy**: 90.00%
- **Test Accuracy**: {test_accuracy:.2f}%

## Classes
{chr(10).join([f'{i+1}. {cls}' for i, cls in enumerate(classes)])}

## Python Usage
```python
import torch
from torchvision import transforms
from PIL import Image

Load model
checkpoint = torch.load('best_model.pth')
model = PrototypicalNetwork(embedding_dim=128)
model.load_state_dict(checkpoint['model_state_dict'])
model.eval()

Prepare transform
transform = transforms.Compose([
 transforms.Resize((224, 224)),
 transforms.ToTensor(),
 transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

Load and predict
img = Image.open('food.jpg').convert('RGB')
img_tensor = transform(img).unsqueeze(0)

with torch.no_grad():
 embedding = model(img_tensor)
 # Compare with prototypes for classification
 ...

Mobile Usage
Use `model_torchscript.pt` with PyTorch Mobile for Flutter/React Native.

Citation
If you use this model, please cite:
[Your Name], "A Transformation-Aware Deep Learning Approach for Recognizing Cooked Vegetable Ingredients in Sri

Model Performance

| Metric | Value |
|-----|-----|
| Validation Accuracy | 90.00% |
| Test Accuracy (Few-shot) | {test_accuracy:.2f}% |
| Full Test Accuracy | {full_accuracy:.2f}% |

```